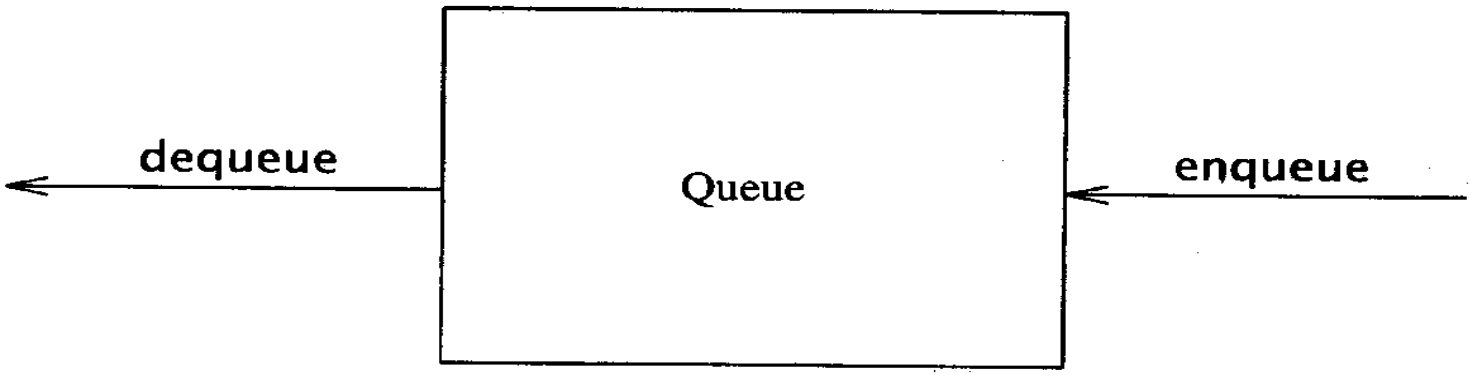
Queue ADT

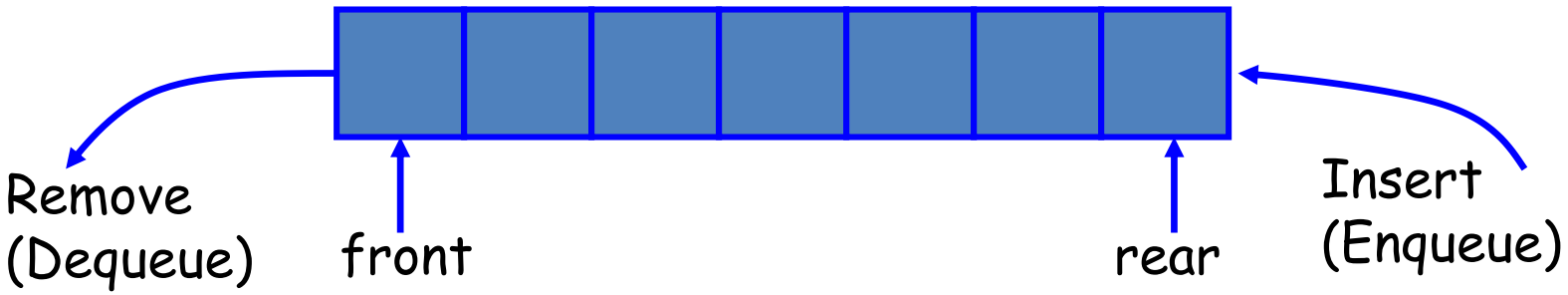
- Like stack, a *queue* is also a list. However, with a queue, insertion is done at one end, while deletion is performed at the other end.

- Basic operations:
  - enqueue: insert an element at the rear of the list.
  - dequeue: delete the element at the front of the list.



Example - Railway reservation line

- Enqueue
  - Insert an element at the rear of the queue.
- Dequeue
  - Remove an element from the front of the queue.



The Queue ADT

- The Queue ADT stores arbitrary objects.
- Insertions and deletions follow the **first-in first-out** scheme.
- Insertions are at the rear of the queue and deletions are at the front of the queue.
- Main queue operations:
  - enqueue(Object o): inserts an element o at the end of the queue
  - dequeue(): removes and returns the element at the front of the queue

- Auxiliary operations:
  - front(): returns the element at the front without removing it.
  - size(): returns the number of elements stored.
  - isEmpty(): returns a Boolean indicating whether no elements are stored.
- Exceptions
  - Attempting the execution of dequeue on an empty queue throws an Exception

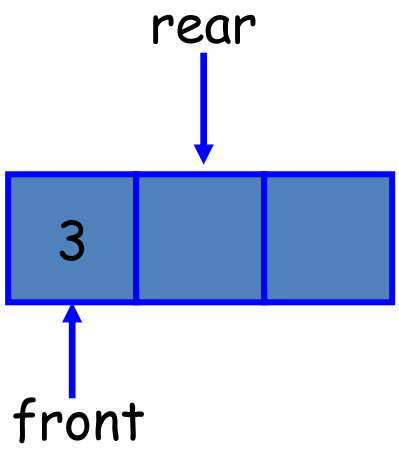
Applications of Queue

- Direct applications
  - Waiting lists, bureaucracy
  - Access to shared resources (e.g., printer)
  - Multiprogramming

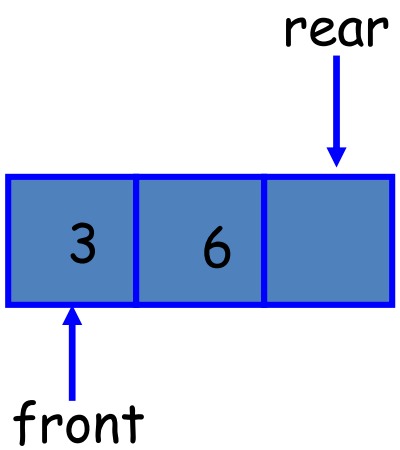
- Indirect applications
  - Auxiliary data structure for algorithms

Array-based Queue Implementation

When enqueuing, the front index is always fixed and the rear index moves forward in the array.



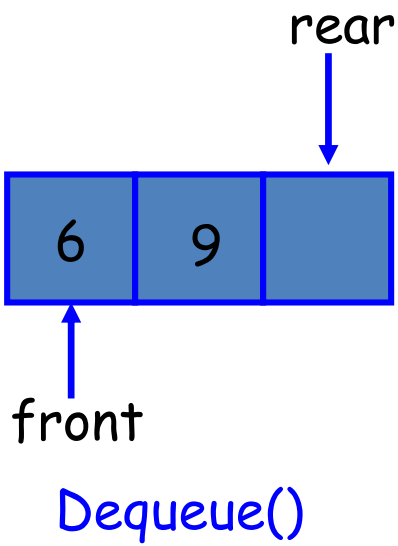
Enqueue(3)

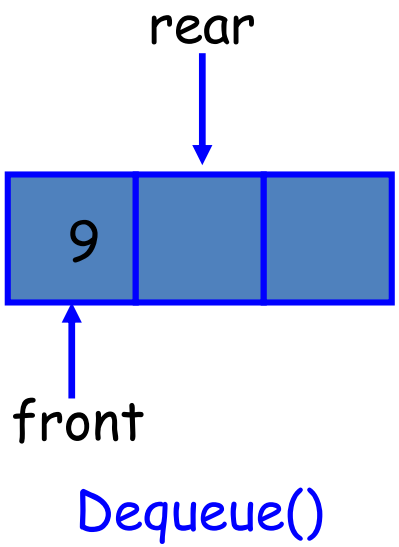


Enqueue(6)

Array-based Queue Implementation

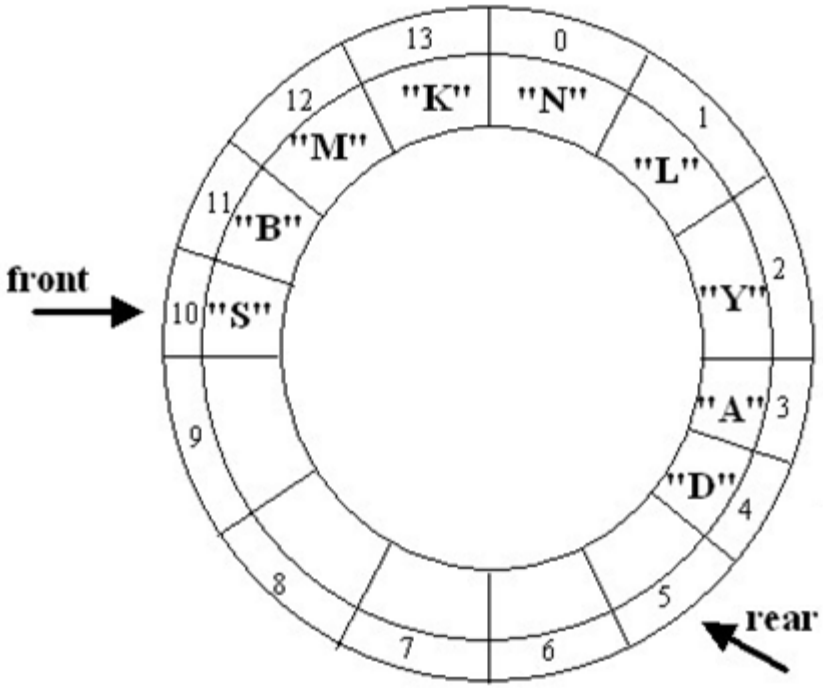
When dequeuing, the element at the front the queue is removed. Move all the elements after it by one position.





Queue As Circular Array Implementation

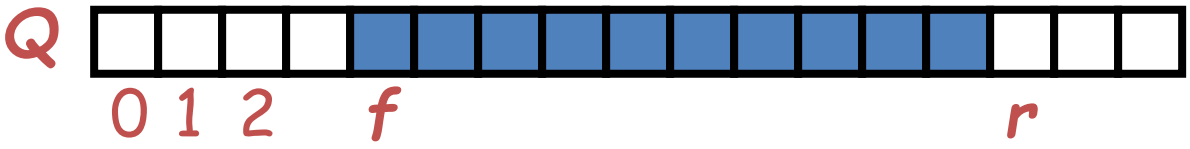
By using modulo arithmetic for computing array indexes, we can have a queue implementation in which each of the operations enqueue and dequeue has complexity O(1).



Queue Operations

```
Algorithm size()
  return (N - f + r) mod N

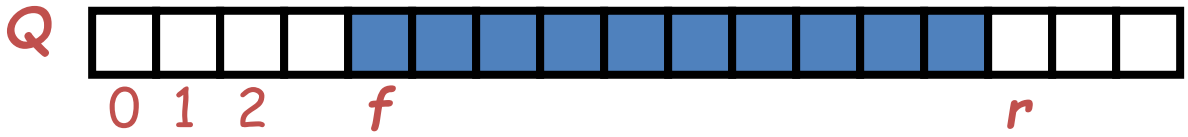
Algorithm isEmpty()
  return true if f = r
```



Queue Operations (cont.)

Operation enqueue throws an exception if the array is full

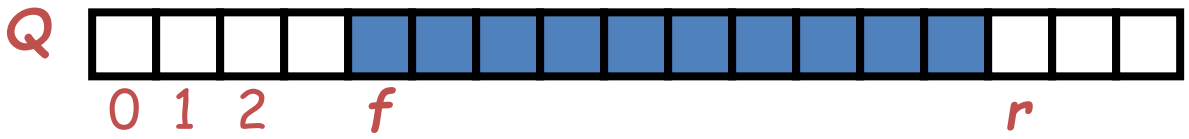
```
Algorithm enqueue(x)
  if size() = N – 1 then
    throw Exception
  else
    Q[r] = x
    r = (r + 1) mod N
```



Queue Operations (cont.)

- Operation dequeue throws an exception if the queue is empty.
- This exception is specified in the queue ADT.

```
Algorithm dequeue()
  if isEmpty() then
    throw Exception
  else
    x = Q[f]
    f = (f + 1) mod N
    return x
```



Growable Array-based Queue

- In an enqueue operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one

- Similar to what we did for an array-based stack

Priority Queue

- In a normal queue the `enqueue` operation add an item at the back of the queue, and the `dequeue` operation removes an item at the front of the queue.

- A priority queue is a queue in which the `dequeue` operation removes an item at the front of the queue; but the `enqueue` operation insert items according to their priorities.

- A higher priority item is always `enqueued` before a lower priority element.

- An element that has the same priority as one or more elements in the queue is `enqueued` after all the elements with that priority.